

Chapter 2: Database System Concepts and Architecture

Elmasri & Navathe — Exam Preparation Notes

Note: This chapter carries lower weightage in exams. Focus on key definitions and the three-schema architecture.

1. Data Abstraction & Data Models

1.1 Data Abstraction

Data abstraction refers to suppressing the details of data organization and storage while highlighting the essential features, so that different users can perceive data at their preferred level of detail. It is one of the main characteristics of the database approach.

1.2 What is a Data Model?

A data model is a collection of concepts used to describe the structure of a database. It provides the means to achieve abstraction and includes:

- **Constructs (elements, groups like entities/tables) and rules that valid data must satisfy at all times.** Structure and Constraints:
- **Basic operations (insert, update, delete) and user-defined operations (e.g., `compute_student_gpa`).** Operations:
- **Computations performed on data, such as calculating a CGPA.** Dynamic Behaviors:

1.3 Categories of Data Models

Category	Description / Examples
Conceptual (High-level / Semantic)	Close to how users perceive data. Also called entity-based or object-based models. E.g., Entity-Relationship (ER) model.
Physical (Low-level / Internal)	Describe how data is physically stored on hardware. Usually specified via DBMS design manuals.
Implementation (Representational)	Fall between conceptual and physical. Used in commercial DBMSs. E.g., Relational model (SQL databases).
Self-Describing	Combine description of data with the data values. E.g., XML, key-value stores, some NoSQL systems.

Exam Tip: The relational model is the most dominant implementation model today. Self-describing models are associated with NoSQL/XML systems.

2. Schemas, Instances, and Database State

2.1 Key Terms

Term	Meaning
Database Schema	The description/structure of a database including data types and constraints. Changes very infrequently. Also called intension.
Schema Diagram	Visual display of the database schema (e.g., Figure 2.1 showing tables like STUDENT, COURSE).
Schema Construct	A component of the schema, e.g., STUDENT, COURSE.
Database State / Instance	Actual data stored in the database at a particular moment in time. Also called occurrence or snapshot. Changes every time data is updated. Also called extension.
Initial Database State	The state of the database when it is first loaded.
Valid State	A state that satisfies all structure and constraints of the schema.

Exam Tip: Schema = intension (structure, rarely changes). State = extension (data, changes frequently). This distinction is a common exam question.

3. Three-Schema Architecture

3.1 Purpose

The three-schema architecture was proposed to support:

- Program-data independence
- Support of multiple views of the data

It is not always explicitly implemented in commercial DBMS products but is important for understanding database system organization.

3.2 The Three Levels

Level	Description
External Level (View Level)	Multiple external schemas (views) that describe how different user groups see the data. Usually uses the same data model as the conceptual schema.
Conceptual Level	One conceptual schema describing the structure and constraints for the entire database for a community of users. Uses a conceptual or implementation data model.
Internal Level	One internal schema describing physical storage structures and access paths (e.g., indexes). Uses a physical data model.

3.3 Mappings Between Levels

- **Translates between user views and the conceptual schema.** External/Conceptual Mapping:
- **Translates between the conceptual schema and physical storage.** Conceptual/Internal Mapping:

Programs refer to an external schema. The DBMS maps requests to the internal schema for execution, then reformats the extracted data to match the user's view (e.g., formatting SQL results for a web page).

4. Data Independence

4.1 Types of Data Independence

Type	Definition
Logical Data Independence	Ability to change the conceptual schema without changing external schemas or application programs. Harder to achieve.
Physical Data Independence	Ability to change the internal schema (e.g., reorganizing file structures, adding indexes) without changing the conceptual schema. More commonly achieved.

4.2 How it Works

When a lower-level schema changes, only the mappings between that schema and higher-level schemas need to be updated. The higher-level schemas themselves remain unchanged. Application programs refer to external schemas and therefore do not need to be modified.

Exam Tip: Logical data independence is harder to achieve than physical data independence. Both are enabled by the three-schema architecture.

5. DBMS Languages

5.1 Data Definition Language (DDL)

- Used by the DBA and database designers to specify the conceptual schema.
- In many DBMSs, DDL also defines internal and external schemas (views).
- Some DBMSs use separate languages:
 - SDL (Storage Definition Language) — for the internal schema
 - VDL (View Definition Language) — for external schemas

5.2 Data Manipulation Language (DML)

- Used to specify database retrievals and updates.
- **Specifies what data to retrieve (not how). Set-oriented. Also called declarative languages.**
Example: SQL. High-Level / Non-procedural DML:

- **Retrieves data one record at a time; requires looping constructs.**Low-Level / Procedural DML:
- Can be embedded in a host language (C, C++, Java, COBOL) or used as a standalone query language.

6. DBMS Interfaces

6.1 Types of Interfaces

Interface Type	Description / Example
Stand-alone Query Language	Entering SQL queries interactively (e.g., SQL*Plus in Oracle).
Programmer Interfaces	Embedding DML in programming languages (embedded SQL, JDBC, ODBC, PL/SQL, PHP, Python).
Menu-based (Web-based)	Popular for web browsing.
Forms-based	Designed for naive users filling in entries.
Graphics-based	Point-and-click, drag-and-drop, query by schema diagram.
Natural Language	Requests in written English or speech.
Mobile Interfaces	Transactions via mobile apps.
DBA Interfaces	Creating user accounts, setting parameters, changing schemas.

6.2 Programmer Interface Details

- **e.g., embedded SQL for C/C++, SQLJ for Java.**Embedded Approach:
- **e.g., JDBC (Java), ODBC (Open Database Connectivity) for other languages.**Procedure Call Approach:
- **e.g., Oracle's PL/SQL incorporates SQL and its data types as integral components.**Database Programming Language:
- **PHP (client-side) and Python (server-side) for database programs.**Scripting Languages:

7. Database System Utilities & Tools

7.1 System Utilities

- Loading data from files into databases (includes data conversion tools).
- Backing up the database periodically.
- Reorganizing database file structures.
- Performance monitoring.
- Report generation.
- Sorting, user monitoring, data compression.

7.2 Other Tools

- **Stores schema descriptions, design decisions, user info, usage standards. Active (accessed by DBMS software and users) vs. Passive (accessed by users/DBA only).**Data Dictionary / Repository:
- **Computer-Aided Software Engineering tools. Examples: PowerBuilder (Sybase), JBuilder (Borland), JDeveloper 10G (Oracle).**CASE Tools:

8. Centralized and Client-Server Architectures

8.1 Centralized DBMS

Combines everything (DBMS software, hardware, application programs, user interface) into a single system. Users can connect via remote terminals, but all processing is done at the centralized site.

8.2 Two-Tier Client-Server Architecture

Specialized servers handle specific functions. Clients access servers as needed over a network (LAN, wireless, etc.).

- **Print server, File server, DBMS server, Web server, Email server.**Server types:
- Relational DBMS servers are called SQL servers, query servers, or transaction servers.
- Clients use APIs such as ODBC or JDBC to connect to database servers.
- A client program can connect to several DBMSs (data sources).

8.3 Three-Tier Client-Server Architecture

Common for web applications. Adds an intermediate Application Server (Web Server) layer between the client and the database server.

Tier	Role
Client (Presentation Layer)	User interface and web browser (PC or mobile device). Cannot directly access the database server.
Application/Web Server (Business Logic Layer)	Stores web connectivity software and business logic. Acts as conduit between client and database server.
Database Server (Database Services Layer)	Manages the database and handles queries/transactions.

- **Database server is only accessible via the middle tier, so clients cannot directly access it.**Enhanced Security:

Exam Tip: Two-tier = Client + DB Server. Three-tier = Client + App/Web Server + DB Server. Three-tier is common for web apps and improves security.

9. Classification of DBMSs

9.1 By Data Model

- **Network model, Hierarchical model.** Legacy:
- **Relational model, Object-oriented, Object-relational.** Currently Used:
- **Key-value storage, NoSQL systems.** Recent Technologies:

9.2 Other Classifications

- Single-user vs. Multi-user
- Centralized (single computer, one DB) vs. Distributed (multiple computers, multiple DBs)
- General Purpose vs. Special Purpose (e.g., airline reservation = OLTP)

9.3 Distributed DBMSs (DDBMSs)

- Homogeneous DDBMS
- Heterogeneous DDBMS
- **Loosely coupled with high degree of autonomy.** Federated / Multidatabase Systems:

Modern distributed DBMSs are often client-server based: a set of database servers supporting a set of clients.

9.4 Cost Considerations

- Range from free open-source (MySQL, PostgreSQL) to multi-million dollar configurations.
- Commercial DBMSs offer add-on modules (time-series, spatial, XML) called cartridges (Oracle) or blades.
- Licensing: site license, seat (concurrent users) license, single user license.

10. History of Data Models (Additional Material)

10.1 Network Model

- First implemented by Honeywell in 1964-65 (IDS System). Adopted widely via CODASYL (1971).
- **Models complex relationships; navigational language (FIND, GET, etc.); programmers can optimize navigation.** Advantage:
- **Navigational and procedural; complex pointer arrays; little scope for automatic query optimization.** Disadvantage:

10.2 Hierarchical Model

- Developed jointly by IBM and North American Rockwell (~1965). Led to IBM's IMS, which still has a large customer base.
- **Simple to construct; maps well to naturally hierarchical domains (e.g., org charts); simple language (GET, GET UNIQUE, GET NEXT WITHIN PARENT).** Advantage:
- **Navigational and procedural; database viewed as linear arrangement of records; little query optimization.** Disadvantage:

10.3 Relational Model

- Proposed by E.F. Codd (IBM) in 1970; first commercial systems in 1981-82.
- **DB2, Oracle, MS SQL Server, Sybase, Informix; free versions: MySQL, PostgreSQL.** Examples:
- Currently the most dominant model for database applications.
- **SQL-89, SQL-92, SQL-99, SQL3, and beyond.** Standards:

10.4 Object-Oriented Data Models

- Models for persistent OO programming languages (C++ — OBJECTSTORE, VERSANT; Smalltalk — GEMSTONE).
- Standard: ODMG-93, ODMG 2.0, ODMG 3.0.

10.5 Object-Relational Models

- Trend started with Informix Universal Server.
- Relational systems incorporated OO concepts. Examples: Oracle, DB2, SQL Server.
- Current trend: extending relational DBMSs to process XML, text, and other data types. The term 'Object-relational' is receding in the marketplace.

Quick Reference Summary

Concept	Key Point to Remember
Data Abstraction	Hiding storage details; exposing essential features.
Schema vs. State	Schema = structure (intension, rarely changes). State = data (extension, changes often).
Three Schema Levels	External (views) → Conceptual (whole DB) → Internal (physical storage).
Logical Data Independence	Change conceptual schema without changing external schemas/programs.
Physical Data Independence	Change internal schema without changing conceptual schema.
DDL	Defines schema structure.

DML	Retrieves and updates data. High-level (SQL) = declarative. Low-level = procedural.
Two-tier Architecture	Client + DB Server (ODBC/JDBC).
Three-tier Architecture	Client + App Server + DB Server; common for web apps; enhances security.
ODBC / JDBC	Standard APIs for client programs to access DBMS servers.
NoSQL / Self-describing	Combine data description with data (XML, key-value stores).